

A Large-Scale P2P Botnet Detection Framework via Topology and Traffic Co-Verification

Ziming Zhao¹, Zhaoxuan Li², Tingting Li¹, Fan Zhang^{1,*}

¹Zhejiang University, China

²SKLOIS, Institute of Information Engineering, Chinese Academy of Sciences, China

{zhaoziming, litt2020, fanzhang}@zju.edu.cn, lizhaoxuan@iie.ac.cn

Abstract—Botnets are still serious threats to infrastructure security nowadays. Recently, adversaries tend to leverage peer-to-peer (P2P) manner propagation to construct large-scale botnets since P2P-based schemes have no single points of failure. Over the past few decades, the research and industry communities have proposed a variety of solutions to detect botnets, which mainly involve communication topology identification and network traffic analysis. Yet, coping with the large-scale P2P botnets, the former suffer topology indistinguishability, and the latter struggles under massive background traffic. In this paper, we present TNT, a large-scale P2P botnet detection framework via communication topology and network traffic. As its core, TNT is powered by three tightly-coupled components: (i) tScouter is responsible for profiling the communication topology; (ii) tCommander plans the strategy for node inspection; and (iii) tPatroller investigates the traffic of the corresponding node. Taken together, TNT advances the trade-off between detection accuracy (enhance topology-based results via traffic analysis) and overhead (only check part of node traffic according to the planning). Based on 42 groups of combinations involving 6 types of botnets and 7 legitimate P2P traffic, we perform extensive evaluation and demonstrate that TNT realizes outstanding detection performance, *e.g.*, after checking ~20K nodes, achieve ~99.9% accuracy for a communication graph (including >140K nodes).

Index Terms—Botnet detection, P2P, network topology, reinforcement learning

I. INTRODUCTION

Botnet continues to be a serious threat to the Internet [37], [54], in which adversaries could control hosts to carry out malicious activities such as distributed denial-of-service (DDoS) attacks [51], [29], malware [27], spamming, market manipulation, and so on [59]. In the past decades, the research and industry communities have devoted lots of efforts to detecting botnets [34], [40]. Depending on the detection approaches, prior arts can be divided into traffic-based and topology-based. In traffic-based schemes, security practitioners intend to detect suspected communication behaviors of malicious activities such as command and control (C&C) [16] and domain name system (DNS) queries [2]. While topology-based approaches aim to discover communication patterns from the overlay topologies [35], [58].

*Fan Zhang is the corresponding author. This work was supported in part by National Key R&D Program of China (2023YFB3106800), by National Natural Science Foundation of China (62227805, 62072398, 62172405), by the Natural Science Foundation of Jiangsu Province (BK20220075), by the Fok Ying-Tung Education Foundation for Young Teachers in the Higher Education Institutions of China (No.20193218210004), and by Key R&D Program of Zhejiang Province (2023C01039).

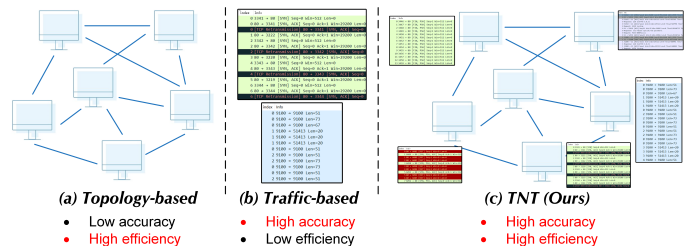


Fig. 1. Illustrative explanation of TNT combining topology and traffic analysis.

According to recent trends, the adversaries amplify the attack surface by introducing large-scale botnets [45], [43], especially leveraging peer-to-peer (P2P) manner propagation since they have no single points of failure [36]. For instance, recent work confirms that Hajime, based on Kademlia Distributed Hash Table (DHT) protocol [32], has infected >100,000 hosts [1], [17], mainly involving IoT devices. When faced with such a large botnet, existing methods encounter some limitations in practice. We summarize the issues as follows, corresponding to Figure 1 (a)-(b).

(i) *Struggling under massive background traffic.* Given the increase in the number of hosts, the number of communications may grow quadratically. What followed could be the explosion of traffic data, especially for widespread high-bandwidth scenarios (*e.g.*, ~100 Gbps) nowadays. However, the communication traffic of botnets usually occupies a small fraction and is hidden in the massive background traffic. Even though traffic detection can provide fine-grained results, analyzing all hosts is impractical due to the high resource consumption.

(ii) *Topology indistinguishability between legitimate P2P services and P2P-based botnets.* The topology-based methods focus on the communication graph, and are indeed not need to analyze per-node traffic, but only using communication patterns to identify botnets is relatively coarse-grained. For example, the Kademlia DHT protocol can be used to create the Hajime botnets (as stated above), while it is also used to distribute data and electronic files over the Internet in a decentralized manner, such as BitTorrent. If only identified based on the communication topology, it may produce a lot of false positives and false negatives. Our experimental results § V-C also confirmed this phenomenon, even though legitimate users and botnets use different P2P protocols.

In this paper, we present TNT, a novel framework that combines Topology and Network Traffic analysis, which aims

to realize high accuracy and efficiency simultaneously, as shown in Figure 1 (c). TNT is designed with three tightly coupled components named as tScouter, tCommander, and tPatroller. First, tScouter is a Graph Neural Network (GNN) model [23], [25] to locate suspicious bot nodes solely based on the communication topology. The output of tScouter is the malicious probability of each node, and these results with the connection relationship of the graph constitute the initial environment for subsequent investigation. Then, tCommander and tPatroller collectively realize the traffic detection for some key nodes. Among them, tCommander is responsible for locating the next node to be detected, and tPatroller performs traffic analysis. Particularly, we design tCommander as a reinforcement learning (RL) based agent that aims to prioritize finding nodes that may be false positives (FPs) or false negatives (FNs). We propose tPatroller as a multi-instance learning model¹ for traffic analysis, the result will update the environment and state for deciding the agent's next action. Through such iterations, TNT can realize superior detection performance, and only detect the few nodes' traffic.

In summary, this paper makes the following contributions.

- We carefully examine the problems that using current traffic-based and topology-based botnet detection in the real world and summarize them as two key challenges. To facilitate addressing the above issues, we propose a co-analyzed framework TNT to combine the two schemes.
- In TNT, we design three tightly coupled components named as tScouter, tCommander, and tPatroller. Among them, tScouter leverages a GNN model to initialize the malicious probability for all nodes. tCommander is responsible for planning the investigative strategy, *i.e.*, determining the next node to be detected. tPatroller will perform traffic analysis for the target node based on a multi-instance learning model. By running tCommander and tPatroller iteratively, high precision and low cost are finally achieved.
- Given that current public datasets lack consideration of P2P background traffic, we utilize OMNet simulation to generate traffic for 7 common P2P protocols. And embed the traffic into the communication graph to produce a dataset that is closer to the real scene for the community.
- Through experiments, we demonstrate that TNT significantly improves the detection accuracy and effect. We release the code and the dataset is available².

II. BACKGROUND AND MOTIVATION

A. Related Work

1) *Communication Topology/Graph Analysis-Based Methods:* Topology-based schemes leverage spatial relationships and unique overlay topology patterns to localize botnet hosts, based on node clustering [35], community detection [20], and graph neural network [58], [31]. For example, Zhou et al. [58] propose

a botnet detection scheme based on Graph Convolutional Network (GCN) to detect the properties of connection structures with different underlying communication patterns. Nevertheless, these arts solely rely on the connection patterns may cause them to be indistinguishable between P2P botnets and legitimate P2P hosts, leading to lots of false positives and false negatives.

2) *Traffic Detection-Based Methods:* Traffic detection [57] is an effective technique that can characterize interaction behaviors of botnet nodes. Previous works including signature-based [14], [16], payload-based [14], and flow-based [38], [50] approaches. Since recent botnets tend to use encryption traffic to hide their payload information, payload-based approaches could be circumvented by attackers and lose effectiveness, and signature-based methods are similar.

To this end, flow-based methods identify botnets' behaviors based on packet field features (*e.g.*, packet length sequences), and traffic representations are distinguishable between P2P bots and legitimate P2P applications [59], [46], [55]. Although these methods enable identifying malicious botnet behavior, detecting all-host traffic in large-scale wide area networks (WANs) remains time-consuming and expensive (*e.g.*, labor or computing resource overhead). This can be attributed to two facts: (i) the large number of hosts in the network topology and (ii) the massive legitimate background traffic in each host.

3) *Host Defense-Based Methods and Others:* There are also a series of schemes to capture anomalous behavior of botnets such as triple-channel botnets [6] and Hybrid botnet [44] by analyzing device logs [13], honeynets monitoring [42], performing reverse-engineering [5], malware program analysis [24], and inferring protocol state machines [8]. Unfortunately, these approaches are vulnerable to malware obfuscation or polymorphism, *e.g.*, the call orders of malware actions can be changed/confused by adversaries. Also, most of them require host-side installation, *e.g.*, anti-virus software and virtual machine, limiting the effect against large-scale botnets.

Different from existing solutions, to detect botnet hosts in the large topology network accurately and quickly, we initially identify suspicious botnet hosts based on the topology/graph features of the communication network. The initial results may produce some false positives and false negatives due to topology indistinguishability. Then, reinforcement learning is leveraged to preferentially examine nodes that could be FPs or FNs based on current-environment knowledge. For the host to be examined, we will perform traffic analysis based on the tailored multi-instance learning. In this way, our scheme aims to achieve accurate detection results while ensuring efficiency.

B. Motivation

(i) *Leveraging communication topology is promising in large-scale botnet detection but lacks fine-grained distinguishability.* Considering that botnets usually have certain communication patterns whether decentralized P2P or centralized C&C. For instance, a series of popular structured botnets present a fast mixing process that represents the convergence track of random walks to a stationary distribution [35]. Also, the detection difficulty of various protocols is more or less different given

¹The advantage is that does not require session-level labels and thus is practical in the real world.

²Online repository <https://github.com/Secbrain/TNT/>

that their graph average path lengths vary [58]. In general, leveraging topology is promising and efficient in large-scale botnet detection since the communication graph data is far less than the traffic data. However, the topology data lacks some fine-grained information, such that it could not distinguish legitimate P2P nodes and P2P-based botnets given they have similar communication graphs.

(ii) *Characterizing network traffic could detect fine-grained behavior while it is impractical to analyze the traffic of all nodes.* To build a botnet, the adversaries usually exploit some vulnerabilities (e.g., remote exploitation or default credentials) to invade victim devices, then the attacker will transmit malicious scripts to gain control of compromised devices [2], [17], such as command & control (C&C). The aforementioned intrusion processes will generate corresponding network traffic, and there is a lot of research on detecting these attacks from network traffic [14], [15], [16]. However, with the development of network infrastructure and the popularization of $\sim 100\text{Gbps}$ or even $\sim \text{Tbps}$ bandwidth, the botnet traffic could be submerged in massive background traffic (large-scale overlay topologies). Therefore, although traffic can provide precise behavior detection, the large overhead limits its practices in the real world.

(iii) *Co-analyzing topology and traffic by reinforcement learning planner.* By comparison, detecting traffic is more accurate and fine-grained but struggles with massive data, while communication topology analysis is the opposite. To reconcile these issues, we propose co-analyzing and deploying a reinforcement learning (RL)-based agent to plan the intelligent detection strategy. In such a process, the agent will determine the nodes that need to be priority detected (often false positives or false negatives) according to the topology and current environment information, thereby significantly improving the accuracy but only analyzing traffic from a small fraction of the node. Ultimately, high precision and low cost are achieved simultaneously.

III. ASSUMPTIONS AND PROBLEM SPACE

Threat Model and Assumptions. We consider that strong adversaries adopt common P2P protocols to transmit their malicious scripts and build botnets. Meanwhile, this P2P protocol traffic/communication could have appeared in a series of legitimate services. For real-world consideration, we consider different churn and routing modes in P2P communities. Among them, we set lifetime churn [10] and no-churn on churn mode, as well as iterative and semi-recursive for routing types. For example, lifetime churn means the host may come online during the day but could be shut down at night. Particularly, considering that labeling each traffic session is labor-intensive, we intend to advance the multi-instance learning model for bag-level analysis, which is more consistent with handling large-scale traffic in real-world scenarios. Moreover, diverse P2P protocols are considered, and we do not assume additional collaborations from other Internet entities, such as IP blacklists provided by security vendors.

Problem Space. For large-scale P2P botnet detection, we tend to achieve the following main goals: (i) High identification precision and (ii) Low detection overhead. The former refers to maintaining low false positives and false negatives when existing legitimate P2P flow in the background traffic (even the botnet/benign nodes use the same P2P). The latter indicates inspecting the traffic as a small fraction of nodes as possible, thereby coping with the large-scale network topology. Overall, the main idea of design TNT is to advance the advantages of both topology-based and traffic-based schemes, realizing a trade-off between accuracy and efficiency.

IV. DESIGN DETAILS OF TNT

A. Architecture and Overview

In Figure 2, we depict a high-level architecture of TNT, including three key components and the detection pipeline. Among them, tScouter locates suspicious botnet nodes based on communication topology, tCommander uses the tScouter's output as the initial environment to decide the next node to be detected, and tPatroller will analyze the corresponding network traffic. The detection result of tPatroller updates the environment, and tCommander continues to make decisions based on the new state. This iterative process can correct false positives and false negatives to improve accuracy.

B. tScouter: Recognizing Communication Topology

1) *Topology Graph Representation:* A communication topology (i.e., graph) can be defined as $\mathcal{G} = \{\mathcal{V}, A\}$, where $\mathcal{V}$ is the host set consisting of n nodes $\{v_1, \dots, v_n\}$, and $A \in \mathbb{R}^{n \times n}$ is a symmetric matrix used to characterize the adjacency relationship for nodes. Among the matrix, $a_{ij} = 1$ indicates an edge between nodes v_i and v_j , and 0 otherwise. To record the edge number of nodes, the node degree matrix is defined as $D = \text{diag}(d_1, \dots, d_n)$ with $d_i = \sum_{j=1}^n a_{ij}$.

$$A = \begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{n1} & \cdots & a_{nn} \end{pmatrix}, D = \begin{pmatrix} d_1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & d_n \end{pmatrix} \quad (1)$$

2) *Node Identification:* We employ a Graph Convolutional Network (GCN) [23] model for tScouter to identify suspicious nodes. To perform node classification, GCN can represent each node vector of size f (i.e., the dimension of node input) as a vector of size c (i.e., the number of classifications) through multiple graph convolutional layers. At the final layer, the node vectors are used to predict their properties, in our case refer to the probability of belonging to botnets. Formally, let $X^{(l)} = \{x_1^{(l)}, \dots, x_n^{(l)}\} \in \mathbb{R}^{n \times c}$ ($l \in \{1, \dots, L\}$) denote the node feature matrix after layer l and the row feature vector of node i represented as $x_i^{(l)} = \{h_0, \dots, h_{c-1}\} \in \mathbb{R}^c$ (especially, $x_i^{(0)} \in \mathbb{R}^f$). These vectors are updated at every layer, so as to construct a hierarchical profile with higher-level vectors representing broader and more abstract properties.

At each GCN layer, the vector of each node is first transformed by a learned matrix $W^{(l)} \in \mathbb{R}^{n \times c}$ as $\tilde{x}_i^{(l)} = x_i^{(l-1)} W^{(l)}$. Then each node's representation vector is averaged with that

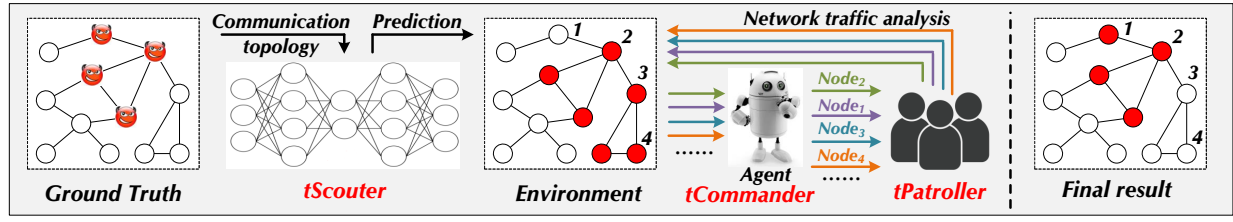


Fig. 2. The overview of TNT. tScouter analyzes the communication topology to generate the initial environment. tCommander and tPatroller are performed iteratively, the former determines the inspected node and the latter performs traffic identification.

of its direct neighbors, which allows this node to include its neighbor information, as a way to effectively explore the topology structure.

$$\begin{aligned} x_i^{(l)} &= \bar{A}_i X^{(l-1)} W^{(l)} = (D^{-1} A)_i W^{(l)} \\ &= \sum_{j=1}^n \frac{a_{ij}}{d_i} \tilde{x}_j^{(l-1)}, \forall i \in \{1, \dots, n\} \end{aligned} \quad (2)$$

where the random walk style normalization (*i.e.*, $\bar{A} = D^{-1}A$) is used to prevent numerical instabilities for GCN models. Furthermore, a non-linear function σ is applied at the end to complete updating the node representation vectors for the current layer. More compactly, we can express the above update in matrix form as $X^{(l)} \leftarrow \sigma(\bar{A}X^{(l-1)}W^{(l)})$, where σ usually employs ReLU activation function (*i.e.*, $\sigma(x) = \max(0, x)$). We also have a separate linear transformation to map the representation vectors in the last layer,

$$X^{(l)} \leftarrow \sigma(X^{(l-1)}U^{(l)} + \sigma(\bar{A}X^{(l-1)}W^{(l)})) \quad (3)$$

where $U^{(l)}$ is a learnable transformation matrix at layer l . Finally, the top node representations $X^{(L)}$ (after L layers) are cascaded with the softmax function, and the prediction vector of node i can be denoted as follows.

$$\mathcal{C}_i = \text{softmax}(x_i^{(L)}) = \frac{e^{h_k}}{\sum_{k=0}^{c-1} e^{h_k}} \quad (4)$$

where $\mathcal{C}_i$ refers to classification confidence and $\sum \mathcal{C}_i = 1$. We take out the second dimension of $\mathcal{C}_i[1]$ that indicates the suspicious probability of belonging to botnets and initial the state table of subsequent tCommander. Particularly, we could set a cross-entropy (*i.e.*, $\arg \max(\mathcal{C}_i)$ refers class) as the loss function during the training phase. And the first layer input is fed to all ones with $X^{(0)} = \mathbf{1} \in \mathbb{R}^{n \times 1}$, as we tend feature initialization agnostic to any ordering of nodes for purely topological learning.

C. tCommander: Planning Inspection Strategy

Once tScouter outputs the prediction confidence for every node, we could get referable results about the malicious probability of each host. However, there will be a large number of false positives and false negatives only based on the communication topology since the similar connection relationship of the P2P protocol for both botnets and legitimate services. A strawman design to mitigate precision loss would be detecting traffic for the node within a specific confidence range or randomly, while this type of scheme cannot benefit

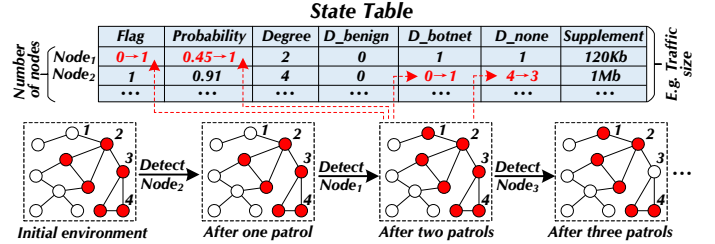


Fig. 3. The state table design.

from the information about node neighbors and corresponding connections. In this section, we will elaborate on the proposed strategy of training the agent to make decisions based on reinforcement learning.

1) *Environment and State*: The environment refers to a series of information sets and the state as input will be fed to agent.

Environment. In our problem space, the environment mainly includes a node list, edge list, and label set. Among them, the node list records the host index, and the label set stores the corresponding ground truth (“benign” as “0” and “botnet” as “1”). Particularly, we use the neighbor list to store edge information given two reasons. On the one hand, it imposes less space overhead than the adjacency matrix given there are $\sim 140K$ nodes and $\sim 1500K$ edges (details in § V-A). On the other hand, it is more efficient than the two-tuple edge list to find neighbors to update state information (introduced later) during the iterative process.

State Table. As the input of subsequent models, the state should be able to represent the current topology information and will be updated during iteration. To this end, we develop the state table structure as shown in Figure 3. It consists of 7-dimension properties, *i.e.*, “Flag”: records whether it has been reviewed; “Probability”: refers to the confidence (initialized by tScouter and will be updated with the detection result); “Degree”: indicates the degree of this node (undirected); “D_benign”: represents the number of viewed benign neighbors of this node; “D_botnet”: represents the number of viewed botnet neighbors of this node; “D_none”: represents the number of unreviewed neighbors. Therefore, “Degree” = “D_benign” + “D_botnet” + “D_none”. For the last dimension, it stores some additional node features for building heterogeneous scenes, *e.g.*, traffic size of this node.

2) *Agent and Action*: To decide on the next node to check, we design a tailor-made neural network architecture as shown in Figure 4(a). Specifically, the agent’s input (*i.e.*, state table) is a $N_n \times N_s$ matrix, where N_n represents the number of

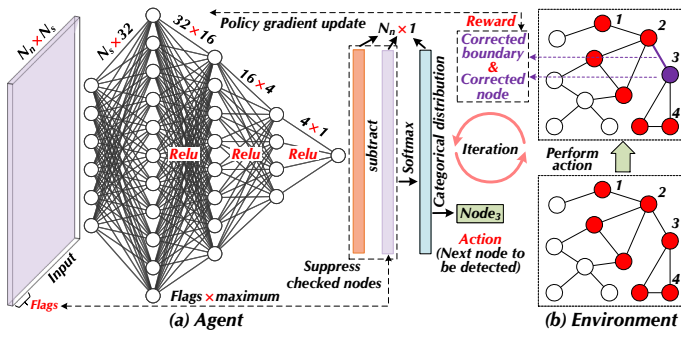


Fig. 4. The model architecture for agent.

nodes and N_s denotes the dimension of state table (e.g., 7). Then, it is concatenated with four Linear layers (i.e., $N_s \times 32$, 32×16 , 16×4 , 4×1) and three ReLU activations, which the corresponding feature maps are $N_n \times 32 \rightarrow N_n \times 16 \rightarrow N_n \times 4 \rightarrow N_n \times 1$. Subsequently, the $N_n \times 1$ vector (V_o) will subtract the product of the maximum (e.g., *float.max*) and flag vector V_t (from input). Also, this process can suppress the probability of selecting the node that has already been detected. Finally, that difference vector is cascaded with the softmax layer and sampled by “Categorical distribution” (enrich the search space) to get the next node to be detected, as follows:

$$\text{Action} = \text{Categorical}(\text{Softmax}(V_o - V_t)) \quad (5)$$

3) *Reward and Training*: When the agent makes a decision, tPatroller will detect the traffic of the corresponding node and provide feedback on whether the host has botnet behavior. The detection results will update the environment and state tables. Meanwhile, the reward value will be calculated to update model parameters during the training process. We leverage the Policy Gradient method [41] to optimize the agent, and the specific details are following.

Policy Gradient. The goal of the RL agent is to find the policy π , which maximizes the expected value of the immediate rewards $\mathcal{R}$. To this end, it will execute a series of actions $\mathcal{A}$ from states $\mathcal{S}$ based on transition probability $\pi(\mathcal{A}|\mathcal{S})$. The policy gradient methods aim to optimize the policy directly, and this expected value is the cost-to-go function as Eq. (6).

$$J(\theta) = \sum_{s \in \mathcal{S}} p^{\pi_\theta}(s) V^{\pi_\theta}(s) = \sum_{s \in \mathcal{S}} p^{\pi_\theta}(s) \sum_{a \in \mathcal{A}} \pi_\theta(a|s) Q^{\pi_\theta}(s, a) \quad (6)$$

where $V^{\pi_\theta}(s)$ represents the value function of the state s , $p^{\pi_\theta}(s) = \sum_{t=0}^{T-1} \gamma^t p(s_k = s | \pi_\theta)$ is the stationary distribution of Markov chain for π_θ , and $Q^{\pi_\theta}(s, a) = \mathbb{E}[\sum_{t=0}^{T-1} \gamma^t r_{t+1} | s_0 = s, a_0 = a, \pi_\theta]$. Given the policy gradient theorem [22], the gradient computation $\nabla_\theta J(\theta)$ can be simplified as follows.

$$\nabla_\theta J(\theta) \propto \mathbb{E}_{\pi_\theta} \left[\left(\sum_{t'=t+1}^T \gamma^{t'-t-1} r_{t'} \right) \nabla_\theta \ln \pi_\theta(a_t | s_t) \right] \quad (7)$$

$$\theta^{\text{new}} = \theta^{\text{old}} + \eta \sum_{t=0}^{T-1} \nabla_\theta J(\theta_t) \quad (8)$$

The iterative process of gradient descent for tCommander is defined in Eq. (8). Note that we feedback the gradient for the model at each T time step, thus guiding it to discover

the error-reported nodes in an efficient manner. Furthermore, the reward r_t of checking a node consists of two parts: (i) node correction and (ii) boundary discovery (such as the purple mark in Figure 4), thus motivating tCommander to prioritize tScouter misjudged nodes rather than those correctly identified. Specifically, the reward r_t is calculated as follows.

- $r_t = \#\{\text{Prob}\} Q_{\text{node}} + \#\{\text{D_bot}\} Q_{\text{edge}}$, if $\#\{\text{Prob}\} - \text{label} \geq 0.5$;
- $r_t = (1 - \#\{\text{Prob}\}) Q_{\text{node}} + \#\{\text{D_ben}\} Q_{\text{edge}}$, if $\text{label} - \#\{\text{Prob}\} > 0.5$;
- $r_t = 0 + \text{minimum_value}$, otherwise.

Among them, $\#\{\text{Prob}\}$, $\#\{\text{D_bot}\}$, and $\#\{\text{D_ben}\}$ refer to the items “Probability”, “D_botnet”, and “D_benign” of the checked node in the state table. Also, label represents its ground truth, and discovering the true predicted nodes will not produce the rewards, i.e., $r_t = 0 + \text{float.min}$. Thus, tCommander is delivered to find more error nodes to earn generous rewards. In addition, Q_{node} and Q_{edge} are the unit rewards of the node correction and boundary discovery, respectively. In the experiment, we set $Q_{\text{node}} = 1$ and $Q_{\text{edge}} = 0.5$ to balance their quantitative differences.

Termination Condition. Through the above process, tCommander can locate the nodes mispredicted by tScouter based on the current state table. Eventually, tCommander completes training on one sample when (i) all the nodes are inspected (i.e., all the “Flag” values in the state table are 1), and (ii) the rewards for $\mathcal{N}_T$ (e.g., 100) consecutive searches are all close to 0. Also, the termination condition (i) is applied to the testing stage. Note that users can early stop patrolling for desired results within limited node queries.

D. tPatroller: Investigating the Node Traffic

tPatroller will identify the class of nodes specified by tCommander based on the nodes’ communication traffic. Specifically, (i) tPatroller collects all communication traffic related to the checked node $\mathcal{N}_i$, including it as the sender or receiver. (ii) These flows $\mathcal{T}_{\mathcal{N}_i}$ will be divided according to session and their features are extracted as identified sample sets $\mathcal{M}_{\mathcal{N}_i} = \{m_1, \dots, m_n\}$. (iii) tPatroller detects these samples and obtains their predicted results $\mathcal{C}_{\mathcal{N}_i} = \{c_1, \dots, c_n\}$. The final class of $\mathcal{N}_i$ is botnet when it contains botnet flows (i.e., $\exists i \in [1, n], c_i = 1$), and benign otherwise. If the identified result is not the same as the class predicted by tScouter, the node can be considered an error node, and its label will be corrected. Considering the above requirements, we design tailor-made multi-instance learners for traffic analysis, and the details are as follows.

Traffic Preprocessing and Feature Extraction. For the checked node $\mathcal{N}_i$, the passing traffic can be captured based on the *tshark* API. For given traffic, we extract the per-packet feature fields including “Protocol”, “Packet Length”, “Time To Live”, “Timestamp”, “TCP Flags”, and “Window Size”, as shown in Figure 5.

Stacking and Bagging. (i) Stacking: for per-packet feature fields (1×6), we stack them according to the 5-tuple index (i.e., {Source IP, Source Port, Destination IP, Destination

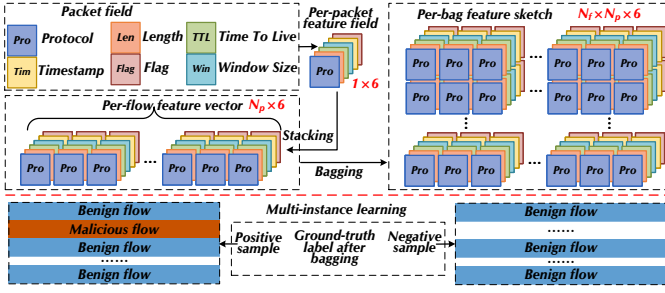


Fig. 5. Multi-instance learning of network traffic analysis.

TABLE I
DATASETS USED IN OUR EVALUATION.

Back. & Bot.	Bot. Nodes	Avg Nodes	Avg Edges
Chord	10,000	143,898	1,502,858
Debru	10,000	143,898	1,672,793
Kadem	10,000	143,898	1,522,895
Leet	10,000	143,898	1,511,483
C2	3,210	143,898	1,626,571
P2P	3,091	143,898	1,623,331

Settings	Ben. P2P	Avg Nodes	Avg Edges
Churn LifetimeChurn & NoChurn	Bamboo	10,190	2,081,491
	Broose	10,190	1,332,513
	Chord	10,190	971,914
	Gia	10,190	215,326
Routing Iterative & Semi-recursive	Kademlia	10,191	1,275,399
	Koorde	10,191	429,511
	Nice	10,191	972,215

Port, Protocol}) to generate the per-flow feature vector. All flow feature vectors will be pruned to the same length N_p by intercepting and zero padding, where N_p is a pre-set hyperparameter. (ii) Bagging: for per-flow feature vectors of $N_p \times 6$, they will be bagged to $N_f \times N_p \times 6$ matrix, *e.g.*, based on source IP. As shown in Figure 5, after flow bagging, the ground-truth label refers to: “malicious” if there exists malicious flow; otherwise, it is “benign”. This bagging process can greatly reduce the calculation time for a batch of inputs due to matrix vectorization and broadcasting. More importantly, this allows practitioners to perform bulk labeling for the sample set, given labeling each traffic session is labor-intensive.

Traffic Identification with Multi-Instance Model. As mentioned above, N_f could be a variable given traffic bagging involves multiple sessions. Let $X = \{x_1, x_2, \dots, x_{N_f}\}$ is a bag of sessions (so-called instances), $y_k \in \{0, 1\}$ ($k \in [1, N_f]$) is the label of session $x_k \in \mathcal{R}^{N_p \times 6}$, and $Y = \max_{k \in [1, N_f]} \{y_k\}$ is that of the bagging sample. Leveraging a class of transformations by a neural network that transforms the instance into a low-dimensional embedding. Then, the multiple-instance learning (MIL) pooling [18] calculates the weight of the instance. Finally, the representation is generated to provide mass information for the bag-level classifier. For example, it could be able to find key instances, *i.e.*, the positive instance may hold the higher weights. During model training, we employ the cross entropy as the loss function of the model.

V. EVALUATION

In this section, we evaluate TNT in terms of traffic detection, topology detection, and integration detection results.

TABLE II
THE USED TRAFFIC DATASETS.

ISCX Botnet dataset					
Type	Packet	Type	Packet	Type	Packet
Benign	3,434,288	Neris	57,374	TBot	37,727
Blackhole	15,521	Osx_trojan	367	Virus	73,301
IRC	41,271	RBot	75,995	Weasel	42,860
Menti	24,387	Smoke-bot	2,710	Zeus	15,653
Murlo	44,067	Sogou	20,604	Zero-access	19,740

Benign background of CAIDA

TABLE III
THE TRAFFIC DETECTION EVALUATION.

Model	Acc (%)	Pre (%)	Rec (%)	F1 (%)
Kitsune	98.52	94.22	91.08	92.62
Whisper [11]	98.79	95.63	92.33	93.95
ProGraph [26]	99.01	96.19	94.02	95.09
ERNN [48]	99.21	97.10	95.12	96.10
CMD [47]	99.59	98.25	97.75	97.99
FS-Net [30]	98.98	96.08	93.82	94.94
ET-BERT [28]	99.72	98.58	98.63	98.60
Ours (session)	99.78	98.89	98.92	98.91
Ours (bag)	98.87	98.76	98.86	98.81

A. Experimental Setup

Datasets. The datasets used for evaluation are summarized in Table I, including botnet communication graphs from Zhou *et al.* [58] and 7 common P2P traffic generated based on OverSim [3]. The former produces four P2P-based botnets topologies (*i.e.*, Koorde [21], Kademlia [32], Chord [39], Leet-chord [19]) and collects two real-world botnets (*i.e.*, P2P and C2 [12]), then overlays these six types of communication graphs into the background traffic from the IP backbone of CAIDA 2018 [7]. For the latter, we generate traffic of 7 common P2P protocols (*i.e.*, Bamboo, Broose, Chord, Gia, Kademlia, Koorde, and Nice) based on OverSim, given the lack of consideration of legitimate P2P traffic in previous literature. Moreover, since the P2P protocols usually are highly resilient that the P2P node could churn in the real world [4]. So we consider different churn types (lifetime churn [10] and no-churn) and routing types (iterative and semi-recursive) in each P2P protocol. We conduct large-scale synthetic communication graphs including botnet and real-world background traffic, with the generated legitimate P2P traffic. The synthesis method, like many previous works [58], [35], employs the node-mapping and embedding scheme. Finally, we produce 42 combinations to test, each group consisting of 96 individual graphs.

For the traffic analysis, we use the ISCX-Botnet [9] dataset in the experiments. As Table II displays, ISCX-Botnet includes benign traffic and 14 types of botnet communication. We also embed legitimate background traffic from CAIDA 2018 [7], to develop large-scale scenarios. If there is no special instruction, data division reference *train : test = 7 : 3*.

Settings. We elaborate here on the parameter settings. Specifically, for the traffic feature map, we set $N_p = 128$. In the multi-learning model, the traffic bagging is randomly produced, the bag length (*i.e.*, N_f) is set to an average of 10 and a standard deviation of 2. Particularly, the number of instances and positive/negative ratio are randomly generated.

TABLE IV

THE CLASSIFICATION RESULTS OF THE SOTA MODEL, AND OUR CALIBRATION RESULTS (WITH 20K RETRIEVALS). “BASE” INDICATES THAT THERE ARE ONLY P2P BOTNETS AND NO BENIGN P2P USERS.

Botnet	Chord								Koorde							
	Acc	FPR	FNR	Rec	Pre	F1	AUC	Our Acc	Acc	FPR	FNR	Rec	Pre	F1	AUC	Our Acc
Base	99.87	0.03	1.41	98.59	99.58	99.08	99.98	-	99.99	0.00	0.10	99.91	99.95	99.93	99.99	-
Bamboo	92.23	7.00	18.11	81.89	46.62	59.42	92.76	99.46 ▲7.22	94.06	6.28	1.46	98.54	53.95	69.72	95.13	100.0 ▲5.94
Broose	92.31	6.99	17.09	82.91	46.96	59.95	92.81	99.54 ▲7.24	93.39	7.04	0.88	99.12	51.25	67.57	93.38	100.0 ▲6.61
Chord	92.55	7.02	13.23	86.77	47.96	61.77	93.03	99.63 ▲7.08	93.54	6.90	0.60	99.40	51.80	68.11	94.63	100.0 ▲6.46
Gia	93.01	7.02	6.54	93.46	49.83	65.00	93.46	99.79 ▲6.78	96.24	4.04	0.03	99.97	64.88	78.69	97.73	100.0 ▲3.76
Kademlia	92.35	7.02	16.06	83.94	47.15	60.38	92.91	99.48 ▲7.13	94.19	6.18	0.89	99.11	54.72	70.44	95.60	100.0 ▲5.81
Koorde	92.81	7.01	9.53	90.47	49.02	63.58	93.30	99.74 ▲6.93	94.43	5.97	0.12	99.88	55.78	71.51	95.99	100.0 ▲5.57
Nice	92.55	7.02	13.23	86.77	47.96	61.76	93.03	99.65 ▲7.10	93.53	6.90	0.62	99.38	51.79	68.10	94.62	100.0 ▲6.47

Botnet	Kademlia								Leet							
	Acc	FPR	FNR	Rec	Pre	F1	AUC	Our Acc	Acc	FPR	FNR	Rec	Pre	F1	AUC	Our Acc
Base	99.86	0.04	1.59	98.41	99.53	98.97	99.96	-	99.91	0.01	1.18	98.82	99.90	99.36	99.98	-
Bamboo	93.05	7.07	5.40	94.60	49.97	65.40	92.97	99.83 ▲6.78	92.90	7.04	7.84	92.16	49.42	64.34	93.16	99.91 ▲7.00
Broose	93.10	7.06	4.84	95.16	50.16	65.69	93.03	99.85 ▲6.75	92.96	7.03	7.06	92.94	49.65	64.73	93.19	99.91 ▲6.95
Chord	93.11	7.11	4.02	95.98	50.19	65.91	93.10	99.87 ▲6.76	93.07	7.06	5.10	94.90	50.06	65.54	93.19	99.93 ▲6.85
Gia	93.17	7.15	2.65	97.35	50.41	66.43	93.52	99.90 ▲6.74	93.29	7.05	2.23	97.77	50.87	66.92	93.53	99.94 ▲6.65
Kademlia	93.09	7.08	4.60	95.40	50.15	65.74	93.04	99.85 ▲6.76	92.99	7.06	6.34	93.66	49.76	64.99	93.17	99.92 ▲6.92
Koorde	93.13	7.16	2.96	97.04	50.28	66.24	93.34	99.89 ▲6.76	93.21	7.06	3.28	96.72	50.56	66.41	93.36	99.94 ▲6.73
Nice	93.10	7.11	3.99	96.01	50.19	65.91	93.10	99.87 ▲6.76	93.07	7.06	5.08	94.92	50.06	65.55	93.18	99.93 ▲6.85

Botnet	P2P								C2							
	Acc	FPR	FNR	Rec	Pre	F1	AUC	Our Acc	Acc	FPR	FNR	Rec	Pre	F1	AUC	Our Acc
Base	99.95	0.02	1.38	98.63	99.25	98.93	99.86	-	99.96	0.00	1.51	98.49	99.83	99.13	99.94	-
Bamboo	92.94	7.02	8.66	91.34	22.62	36.21	93.23	99.97 ▲7.03	99.84	0.00	7.23	92.77	99.92	95.84	99.77	100.0 ▲0.16
Broose	92.93	7.03	8.71	91.29	22.59	36.18	93.22	99.98 ▲7.05	99.76	0.00	10.97	89.03	99.90	94.02	99.86	100.0 ▲0.24
Chord	92.95	7.07	6.23	93.77	22.96	36.84	93.28	99.98 ▲7.03	99.80	0.00	9.30	90.70	99.90	94.93	99.87	100.0 ▲0.20
Gia	93.04	7.07	2.05	97.95	23.76	38.21	93.89	99.99 ▲6.95	99.63	0.33	2.42	97.58	87.20	91.96	99.83	100.0 ▲0.37
Kademlia	92.97	7.06	5.67	94.33	23.10	37.07	93.24	99.99 ▲7.02	99.53	0.30	8.27	91.73	89.36	89.55	99.71	100.0 ▲0.47
Koorde	93.03	7.05	3.40	96.60	23.56	37.84	93.66	99.99 ▲6.96	99.83	0.00	7.88	92.12	99.91	95.81	99.89	100.0 ▲0.17
Nice	92.97	7.07	5.09	94.91	23.18	37.21	93.33	99.99 ▲7.02	99.79	0.00	9.65	90.35	99.90	94.69	99.84	100.0 ▲0.21

Metrics. We adopt a series of metrics, including accuracy *Acc*, false positive rate *FPR*, false negative rate *FNR*, recall *Rec*, precision *Pre*, f1-score *F1*, and area under roc *AUC*.

B. Traffic Detection

We first perform traffic detection, and seven state-of-the-art (SOTA) traffic analysis models are used for comparison. They are Kitsune [33], Whisper [11], ProGraph [26], ERNN [48], CMD [47], FS-Net [30], ET-BERT [28]. Particularly, these baseline methods are based on session-level analysis. For a fair comparison, in addition to bag-level detection, we also report session-level detection results for the proposed tPatroller (it refers to setting the session number of each bag to 1). As summarized in Table III, our model outperforms existing 7 SOTA methods under the same settings (*i.e.*, perform session-level detection). The most competitive algorithm is ET-BERT (*e.g.*, its F1 score is $\sim 0.3\%$ lower than ours), while ET-BERT deploys a pre-trained transformer and is not suitable for handling massive volume of traffic since it is time-consuming.

Nonetheless, our model is capable of analyzing multi-instance traffic (after bagging), which is not possible with previous methods (as stated in § IV-D, the main benefit is that our model does not require session-level labels, which is more practical in the real world). On the last line in Table III, the proposed model still realizes high performance in bag-level analysis (note that the positive/negative ratio is different between session-level and bag-level). For example, it achieves 98.76% precision and 98.86% recall, which is almost comparable to session-level results. Figure 6 shows some

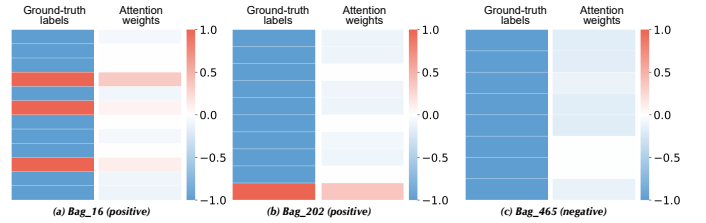


Fig. 6. Some traffic multi-instances after bagging sessions. Among them, “red” denotes the positive sample and “blue” denotes the negative sample.

examples of traffic bagging, and we can see that tPatroller can indeed identify those key sessions with high attention weights, thereby producing correct bag-level outcomes.

C. Communication Topology Detection

To verify topology indistinguishability, we first evaluate the performance of using tScouter to identify suspicious nodes only based on the communication topology. The experimental results for various synthetic combinations are summarized in Table IV. In five P2P-based botnets (*i.e.*, Chord, Koorde, Kademlia, Leet, and P2P), we can see significant drops in various performance metrics by comparing 7 types of benign P2P with the “Base” (no P2P embedded). For instance, the FPR directly dropped 7 percent, noteworthy, given that the embedded legitimate P2P nodes are around 10K (occupying 7.14% of 140K nodes), 7% FPR means $>98\%$ benign P2P users will be misidentified. We also perform an additional experiment group of centralized C2 botnets in the final part of Table. This result is different from P2P-based botnets in that its FPR is less than 0.5% since centralized C2 and decentralized P2P are prone to

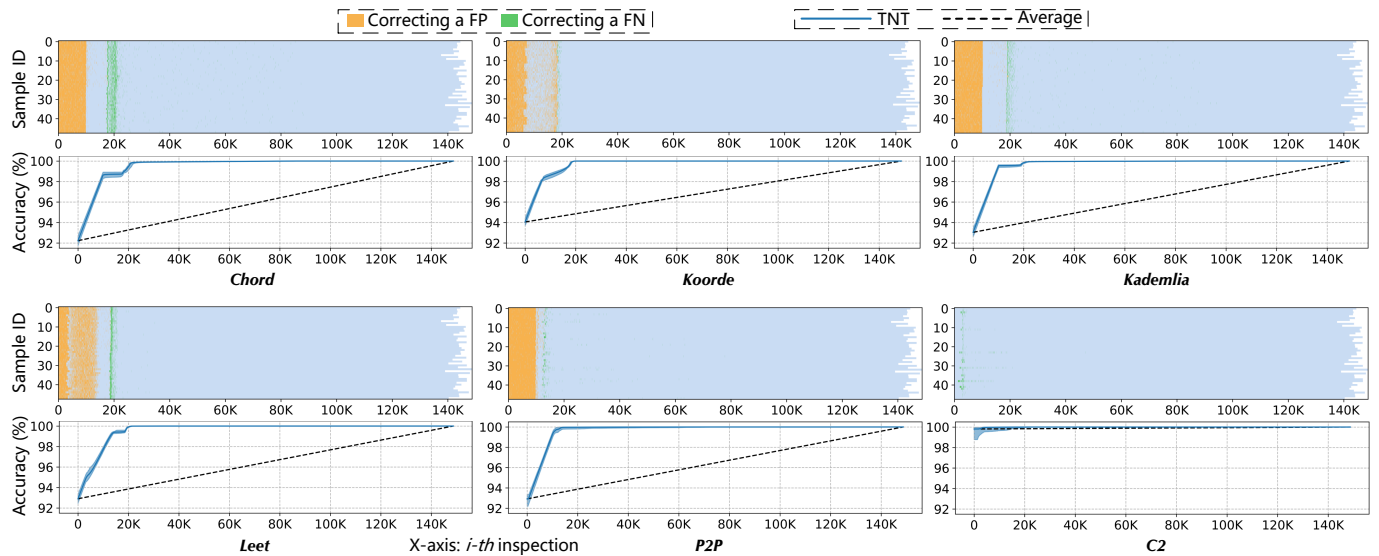


Fig. 7. The top part refers to strategy visualization for each kind of botnet, with the Bamboo-based legitimate P2P traffic. Yellow: correcting a false positive; green: correcting a false negative; blue: correct prediction. The bottom refers to the accuracy growth curve based on TNT.

distinguish. However, the FNR in C2 is relatively high, around 10%. Overall, we confirm the topology indistinguishability between legitimate P2P traffic and P2P-based botnets.

D. Integration Detection

We start the integration test with TNT for 42 combinations.

Strategy Visualization. First, we visualize the detection planning decided by TNT in Figure 7 (more visualization results can be found in the online repository), where the yellow mark refers to correcting a false positive, the green mark means correcting a false negative, and the blue mark denotes correct predictions. As expected, most false positives and false negatives are corrected at an early stage that indicates TNT could quickly locate those nodes that may be wrong. Particularly, there is a series of “jagged” intermittent distributions mainly in Koorde-based botnets. This could be attributed to the correction speeds being different with various churning and routing types.

Performance Enhancement Process. At the bottom of Figure 7, we draw the accuracy growth curve (denote blue line) and the random sampling strategy (black dotted line). For each group of settings, we conduct 10 experiments for TNT to portray the mean, as well as the upper and lower boundaries. It is clear that tCommander can preferentially discover and correct nodes that may be false or false positives. Each experimental group achieves superior performance that TNT realize ACC > 99.9% only using ~20K node inspection. In contrast, without TNT, if choosing the strategy of random sampling, improving the accuracy from ~92% to ~99.9% requires about 140K queries.

VI. DISCUSSIONS

A. Robustness

Considering the possibility of traffic tampering and the existence of adversarial examples [49], traffic analysis models inevitably have misclassification. On the one hand, to ensure

the effect of attack traffic, packet bytes cannot be arbitrarily tampered with, given the traffic needs to meet protocol specification verification. On the other hand, some model robustness solutions [53] can be combined to improve the robustness of the model against adversarial examples.

B. Extensibility

In the evaluation section, TNT can give priority to improving the detection recall and precision, even if against various combinations of P2P protocols. Moreover, we could enrich the state table with more entries, *e.g.*, autonomous system numbers (ASN), active time, etc. For the specific P2P protocol, its communication mode could benefit TNT to further improve performance. For example, one retrofit scheme could consider assimilating neighbor nodes based on the detection results. In other words, when a node has multiple bots in its neighbors, we can consider this node to be a botnet node with high probability. In addition, the traffic analysis module could enhance the ability to identify unknown attacks [52], [56] by profiling outliers.

C. Limitations and Future Works

Our work has a few limitations. First, the current design of TNT mainly considers a single communication graph, we tend to consider scenarios associated with multiple communication graphs in the future. Then, to be more generic in the real world, we could develop a customized construction process to adapt to the various requirements, *e.g.*, prioritize increasing FPR, prioritize traffic size, etc. Finally, as part of future work, we would explore which traffic could be analyzed in parallel to improve efficiency.

VII. CONCLUSION

In this paper, we propose TNT, a novel framework for large-scale P2P botnet detection. It consists of three tightly coupled components, tScouter first initializes node suspicious probability according to communication topology, followed

by tCommander and tPatroller iteratively correcting the possibly wrong nodes by analyzing the corresponding traffic. TNT realizes a trade-off between accuracy and efficiency in an intelligent planning manner. Our experiments demonstrate that TNT could achieve remarkable detection results, only requiring inspecting a small fraction of the node (e.g., $\sim 20\%$). Our research bridges the topology-based and traffic-based perspectives which advance the advantages of both in the botnet detection landscape.

ACKNOWLEDGMENTS

We are grateful to the reviewers for their insightful comments and constructive feedback.

REFERENCES

- [1] O. Alrawi et al. The circle of life: A large-scale study of the iot malware lifecycle. In *USENIX Security Symposium*, 2021.
- [2] M. Antonakakis et al. Understanding the mirai botnet. In *USENIX Security Symposium*, pages 1093–1110. USENIX Association, 2017.
- [3] I. Baumgart et al. Oversim: A scalable and flexible overlay framework for simulation and real network applications. In *Peer-to-Peer Computing*. IEEE, 2009.
- [4] L. Böck et al. Poster: Challenges of accurately measuring churn in P2P botnets. In *CCS*, pages 2661–2663. ACM, 2019.
- [5] J. Caballero et al. Dispatcher: enabling active botnet infiltration using automatic protocol reverse-engineering. In *CCS*. ACM, 2009.
- [6] X. Cai et al. A systematic approach to developing and evaluating website fingerprinting defenses. In *CCS*. ACM, 2014.
- [7] CAIDA. The caida ucsd anonymized internet traces. https://www.caida.org/data/passive/passive_dataset.xml, 2018.
- [8] C. Y. Cho et al. Inference and analysis of formal models of botnet command and control protocols. In *CCS*. ACM, 2010.
- [9] CIC. Botnet dataset. [EB/OL], 2014. <https://www.unb.ca/cic/datasets/botnet.html> Accessed November 27, 2020.
- [10] D. Dagon et al. Modeling botnet propagation using time zones. In *NDSS*, 2006.
- [11] C. Fu et al. Realtime robust malicious traffic detection via frequency domain analysis. In *CCS*, pages 3431–3446. ACM, 2021.
- [12] S. García et al. An empirical comparison of botnet detection methods. *Comput. Secur.*, 2014.
- [13] S. Gianvecchio et al. Measurement and classification of humans and bots in internet chat. In *USENIX Security Symposium*, 2008.
- [14] G. Gu et al. Bothunter: Detecting malware infection through ids-driven dialog correlation. In *USENIX Security Symposium*, 2007.
- [15] G. Gu et al. Botminer: Clustering analysis of network traffic for protocol- and structure-independent botnet detection. In *USENIX Security Symposium*, 2008.
- [16] G. Gu et al. Botsniffer: Detecting botnet command and control channels in network traffic. In *NDSS*, 2008.
- [17] S. Herwig et al. Measurement and analysis of hajime, a peer-to-peer iot botnet. In *NDSS*. The Internet Society, 2019.
- [18] M. Ilse et al. Attention-based deep multiple instance learning. In *ICML*. PMLR, 2018.
- [19] M. Jelasity et al. Towards automated detection of peer-to-peer botnets: On the limits of local approaches. In *LEET*. USENIX Association, 2009.
- [20] H. P. Joshi et al. Improved P2P botnet community detection: Combining modularity and strong community. In *GLOBECOM*. IEEE, 2019.
- [21] M. F. Kaashoek et al. Koorde: A simple degree-optimal distributed hash table. In *IPTPS*. Springer, 2003.
- [22] S. M. Kakade. A natural policy gradient. In *NIPS*. MIT Press, 2001.
- [23] T. N. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. In *ICLR (Poster)*. OpenReview.net, 2017.
- [24] C. Kolbitsch et al. Effective and efficient malware detection at the end host. In *USENIX Security Symposium*, 2009.
- [25] L. Li et al. An automated alert cross-verification system with graph neural networks for IDS events. In *CSCWD*. IEEE, 2024.
- [26] W. Li et al. Prograph: Robust network traffic identification with graph propagation. *IEEE/ACM Trans. Netw.*, 2022.
- [27] Z. Li et al. metanet: Interpretable unknown mobile malware identification with a novel meta-features mining algorithm. *Comput. Networks*, 250:110563, 2024.
- [28] X. Lin et al. ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification. In *WWW*. ACM, 2022.
- [29] X. Ling et al. Ddosminer: An automated framework for ddos attack characterization and vulnerability mining. In *ACNS*. Springer, 2024.
- [30] C. Liu et al. Fs-net: A flow sequence network for encrypted traffic classification. In *INFOCOM*, pages 1171–1179. IEEE, 2019.
- [31] W. W. Lo et al. Xg-bot: An explainable deep graph neural network for botnet detection and forensics. *Internet Things*, 2023.
- [32] P. Maymounkov et al. Kademlia: A peer-to-peer information system based on the XOR metric. In *IPTPS*, Lecture Notes in Computer Science. Springer, 2002.
- [33] Y. Mirsky et al. Kitsune: An ensemble of autoencoders for online network intrusion detection. In *NDSS*, 2018.
- [34] Y. Nadji et al. Beheading hydras: performing effective botnet takedowns. In *CCS*. ACM, 2013.
- [35] S. Nagaraja et al. Botgrep: Finding P2P bots with structured graph analysis. In *USENIX Security Symposium*, 2010.
- [36] C. Rossow et al. Sok: P2PWNEED - modeling and evaluating the resilience of peer-to-peer botnets. In *IEEE Symposium on Security and Privacy*, 2013.
- [37] S. Soltan et al. Blacklot: Iot botnet of high wattage devices can disrupt the power grid. In *USENIX Security Symposium*, 2018.
- [38] Z. Song et al. I2RNN: An Incremental and Interpretable Recurrent Neural Network for Encrypted Traffic Classification. *IEEE TDSC*, 2023.
- [39] I. Stoica et al. Chord: A scalable peer-to-peer lookup service for internet applications. In *SIGCOMM*. ACM, 2001.
- [40] G. Stringhini et al. BOTMAGNIFIER: locating spambots on the internet. In *USENIX Security Symposium*, 2011.
- [41] R. S. Sutton et al. Policy gradient methods for reinforcement learning with function approximation. In *NIPS*. The MIT Press, 1999.
- [42] S. Venkatesan et al. Detecting stealthy botnets in a resource-constrained environment using reinforcement learning. In *MTD@CCS*. ACM, 2017.
- [43] R. Vogt et al. Army of botnets. In *NDSS*, 2007.
- [44] D. Yumei et al. POSTER: A hybrid botnet ecological environment. In *CCS*. ACM, 2014.
- [45] J. Zhang et al. Intention and origination: An inside look at large-scale bot queries. In *NDSS*, 2013.
- [46] J. Zhang et al. Building a scalable system for stealthy p2p-botnet detection. *IEEE TIFS*, 2014.
- [47] Z. Zhao et al. CMD: Co-analyzed IoT Malware Detection and Forensics via Network and Hardware Domains. *IEEE TMC*, 2023.
- [48] Z. Zhao et al. ERNN: Error-Resilient RNN for Encrypted Traffic Detection towards Network-Induced Phenomena. *IEEE TDSC*, 2023.
- [49] Z. Zhao et al. SAGE: steering the adversarial generation of examples with accelerations. *IEEE TIFS*, 2023.
- [50] Z. Zhao et al. Work-in-Progress: Towards Real-Time IDS via RNN and Programmable Switches Co-Designed Approach. In *RTSS*. IEEE, 2023.
- [51] Z. Zhao et al. DDoS family: A novel perspective for massive types of ddos attacks. *Comput. Secur.*, 138:103663, 2024.
- [52] Z. Zhao et al. FOSS: Towards fine-grained unknown class detection against the open-set attack spectrum with variable legitimate traffic. *IEEE/ACM Transactions on Networking*, 2024.
- [53] Z. Zhao et al. Improving model robustness against adversarial examples with redundant fully connected layer. In *WWW (Companion Volume)*. ACM, 2024.
- [54] Z. Zhao et al. Poster: Combine topology and traffic to calibrate P2P botnet identification in large-scale network. In *SIGCOMM (Posters and Demos)*, pages 16–18. ACM, 2024.
- [55] Z. Zhao et al. RIDS: towards advanced IDS via RNN model and programmable switches co-designed approaches. In *INFOCOM*. IEEE, 2024.
- [56] Z. Zhao et al. Trident: A universal framework for fine-grained and class-incremental unknown traffic detection. In *WWW*. ACM, 2024.
- [57] Z. Zhao, Z. Liu, et al. Effective DDoS Mitigation via ML-Driven In-network Traffic Shaping. *IEEE TDSC*, 2024.
- [58] J. Zhou, Z. Xu, A. M. Rush, and M. Yu. Automating botnet detection with graph neural networks. *CoRR*, abs/2003.06344, 2020.
- [59] D. Zhuang et al. Enhanced peerhunter: Detecting peer-to-peer botnets through network-flow level community behavior analysis. *IEEE TIFS*, 2019.